

CityTech TTP Summer Bootcamp

More Express, Pair Programming and Project Structure

2026-06-17

Agenda

1. Review
2. Follow Ups: Yarn, Secrets and `.gitignore`
3. More Express: Query Params, Errors and Middleware
4. Pair Programming and AI
5. Project Structure: Repos and Monorepos
6. Mini Project: Who's That Pokémon?

Review

What are the most important things you remember from yesterday?

Follow Up: Yarn Versions

Two versions of Yarn you'll run into:

- **Yarn Classic (1.x)**: the original; uses a traditional `node_modules/` folder, like npm
- **Yarn Modern (4.x)**: a rewrite; **by default** uses **Plug'n'Play (PnP)** instead of `node_modules`

PnP drops `node_modules` for faster installs and stricter dependency resolution; but some tools expect `node_modules` and break under it.

We'll start with **Classic**: its `node_modules` layout best mirrors **modern npm**, so what you learn is transferable.

Follow Up: API Keys and Secrets

Almost never commit API keys (or passwords, tokens) to version control; once pushed, a secret lives in your git **history forever**, even if you delete it later.

The standard fix:

- Put secrets in a `.env` file
- Add `.env` to `.gitignore` so git never tracks it
- Commit a `.env.example` (the key names, no values) so teammates know what's needed

Example: `.env.example`

Commit a `.env.example` as a template: the key names, but **no real values**.

```
API_KEY=  
DATABASE_URL=  
PORT=3000
```

- Teammates copy it and fill in their own values: `cp .env.example .env`
- The real `.env` is gitignored; the example just documents what's needed

Example: `.gitignore`

A `.gitignore` lists paths git should **not track**:

```
# secrets
.env

# dependencies
node_modules/

# build output
dist/
```

Example: `.gitignore` (continued)

- `.env` : secrets; never commit
- `node_modules/` , `dist/` : **generated** files, rebuilt from your source (`yarn install` , a build step)

Why ignore generated files? They're **derived, huge, and machine-specific**: committing them bloats the repo and causes constant merge conflicts, and anyone can just regenerate them. **Track the source, not the output.**

Express: Query Parameters

```
// GET /sum?nums=2,3,4
app.get("/sum", (req, res) => {
  const nums = req.query.nums.split(",").map(Number);
  res.json({ result: nums.reduce((a, b) => a + b, 0) });
});
```

- Everything after `?` lands on `req.query` as key-value pairs
- **Why not route params here?** They need a *fixed* count: `/add/:a/:b` only ever takes two. To sum five numbers you'd need `/sum/:a/:b/:c/:d/:e`, a new route for every count
- Query params take a **variable, optional** list in one route: `?nums=2,3,4,5,6`
- Like path params, the values arrive as **strings**

Express: Returning an Error

```
app.get("/divide/:a/:b", (req, res) => {  
  const b = Number(req.params.b);  
  if (b === 0) {  
    return res.status(400).json({ error: "Cannot divide by zero" });  
  }  
  res.json({ result: Number(req.params.a) / b });  
});
```

- Choose a fitting **status code** (400 , 404 , ...) and return a message
- **return** so the handler stops

Aside: Method Chaining

`res.status(400).json(...)` calls one method right on the result of the previous - that's **method chaining**.

```
res.status(400).json({ error: "Cannot divide by zero" });
```

- It works because `res.status(400)` **returns** `res`, so you can keep calling on it
- Reads left-to-right as a pipeline - same as writing each call on its own line

Express: Middleware

Middleware is a function Express runs **in between** receiving a request and sending the response.

- Signature `(req, res, next)` : it can read or change `req` / `res` , then calls `next()` to pass control along
- Express runs them in a **pipeline**, top to bottom, **before your route handler**

```
request → [express.json()] → [logger] → [auth] → your route → response
```

- Common ones: `express.json()` , `cors` , a request logger, an auth check
- You register each with `app.use(...)` **once**, usually **near the top** of the file (before your routes)

Middleware: `express.json()`

`express.json()` reads the request **body** and parses JSON into `req.body`.

```
app.use(express.json());           // parse JSON bodies, every request

app.post("/memory", (req, res) => {
  const { value } = req.body;       // available thanks to express.json()
  res.status(201).json({ value });
});
```

- The body arrives as **raw bytes**; Express does **not** parse it on its own, so **without this**, `req.body` is **undefined**
- Needed for requests that **carry a body**: **POST, PUT, PATCH**
- **GET / DELETE** take their input from the URL (`req.params` , `req.query`), so it just **no-ops** for them

Middleware vs. Importing a React Library

Both pull in **reusable code someone else wrote** - often installed the same way (`npm install`, then `import`). The real difference is **who calls it, and when**.

Importing a React library: *you* call it, explicitly, right where you want it:

```
import { format } from "date-fns";  
format(new Date(), "yyyy-MM-dd"); // you decide when this runs
```

Middleware: you **register** it once, and *Express* calls it for you, automatically, on every request. If registering multiple, order matters!

```
app.use(express.json()); // you never call it yourself
```

Middleware vs. Importing a React Library

- **React import** → **you're in control**: you invoke the function at a specific spot in your code
- **Middleware** → **inversion of control**: you hand the function to the framework, and *it* runs it for you - automatically, in a pipeline, before your handler
- A React component renders **where you place it**; middleware runs on **every request** whether or not you "use" it anywhere
- **Order matters for middleware, not for imports**: where you put an `import` is cosmetic, but middleware order changes behavior
- Same idea both ways: don't reinvent it - but middleware plugs into Express's **request lifecycle**, not your own call stack

Pair Programming

Two people, one task, working together in real time.

- **Driver:** types the code
- **Navigator:** reviews, thinks ahead, spots issues, suggests direction
- **Switch roles often**
- Benefits: fewer bugs, shared knowledge, faster unblocking

Fishbowl Demo: Calculator Express Server

Building the calculator **server** with Express, in a **fishbowl**.

- Implement the **add**, **subtract**, **multiply**, and **divide** endpoints

AI as Your Pair Programmer

Treat the AI like a **partner you collaborate with** – not an autopilot.

- **You stay the lead:** set the goal, review every suggestion, decide what to keep
- Use it to brainstorm, explain, draft, and debug – **trust but verify**
- Use it **ethically** – disclose your use, and don't pass off code you don't understand as your own
- *vs. just letting AI code:* you'd ship code you don't understand, can't debug, and can't vouch for
- Rule of thumb: **never commit code you can't explain**

Using CUNY Copilot alongside VS Code

CUNY's licensed AI is **Microsoft Copilot** (data-protected) - it runs in the **browser**, not inside VS Code. So use them **side by side**:

- Open **Copilot Chat** in a browser, signed in with your **CUNY account**
- Keep **VS Code** open beside it
- Copy code / errors into Copilot, paste suggestions back - then **review and verify**
- Because it's CUNY-licensed, your prompts and files **aren't used to train** the model

Heads-up: the in-editor "agent" is GitHub Copilot - a separate product CUNY doesn't license.

Demo: Calculator Client via AI

A quick **demo** of building a calculator **client** with AI as a pair programmer.

- A small **React app** that takes two numbers and an operation
- Calls the Express endpoint with `fetch` (e.g. `/add/3/4`) and shows the result
- Also have the AI generate **standalone Swagger / OpenAPI docs** for the calculator API
- Watch how to **lead** the AI: describe the goal, review the code, **trust but verify**
- ...and disclose the AI use

Multiple Repos vs. Monorepo

Two ways to organize related projects (like our **server** and **client**):

- **Multiple repos (polyrepo):** each project gets its **own git repo**. Independent history, versioning, and deploys; but a shared change spans repos and coordination is harder
- **Monorepo:** **one git repo** holds many projects in subfolders. One clone, one history, easy to change server and client together; but it needs tooling to manage as it grows

Big tech often uses giant monorepos (e.g. Google, Meta).

Structure: Server and Client in One Folder

Put both projects under one parent, each in its **own subfolder** with its **own package.json** :

```
calculator/  
├── server/      # yarn init + express    (the API)  
└── client/      # yarn create vite       (the React app)
```

- Run **yarn init** (or **yarn create vite**) **inside each** subfolder; they install and run **independently**
- This *mimics* a monorepo's layout, but it's **not** a real one: no shared tooling links them (e.g. yarn workspaces)
- Still handy: related code lives together and you develop both side by side

Mini Project: Who's That Pokémon?

Put this week's skills together: an API, an Express server, and a client.

- Full spec: `project_specs/whos_that_pokemon.md`